CS 452 Operating systems

Virtual Memory

Dr. Denton Bobeldyk

Virtual Memory

- Background
- Demand Paging
- Copy-on-Write

Background

- In order for a process to run, it must be in physical memory
- Does the entire process need to be loaded into physical memory?

Background

- No, the entire process doesn't need to be in physical memory
- The program may have error conditions that will never run
- Array lists and tables may be allocated much larger than the data that actually fills them
  - How many times as a programmer did you allocate an array soooo large to make sure it would never fill up?? 😄

Background

- If the entire program actually is needed, will it be needed all at the same time?
- What sort of advantages are there, if we don't have to load the entire program into memory?

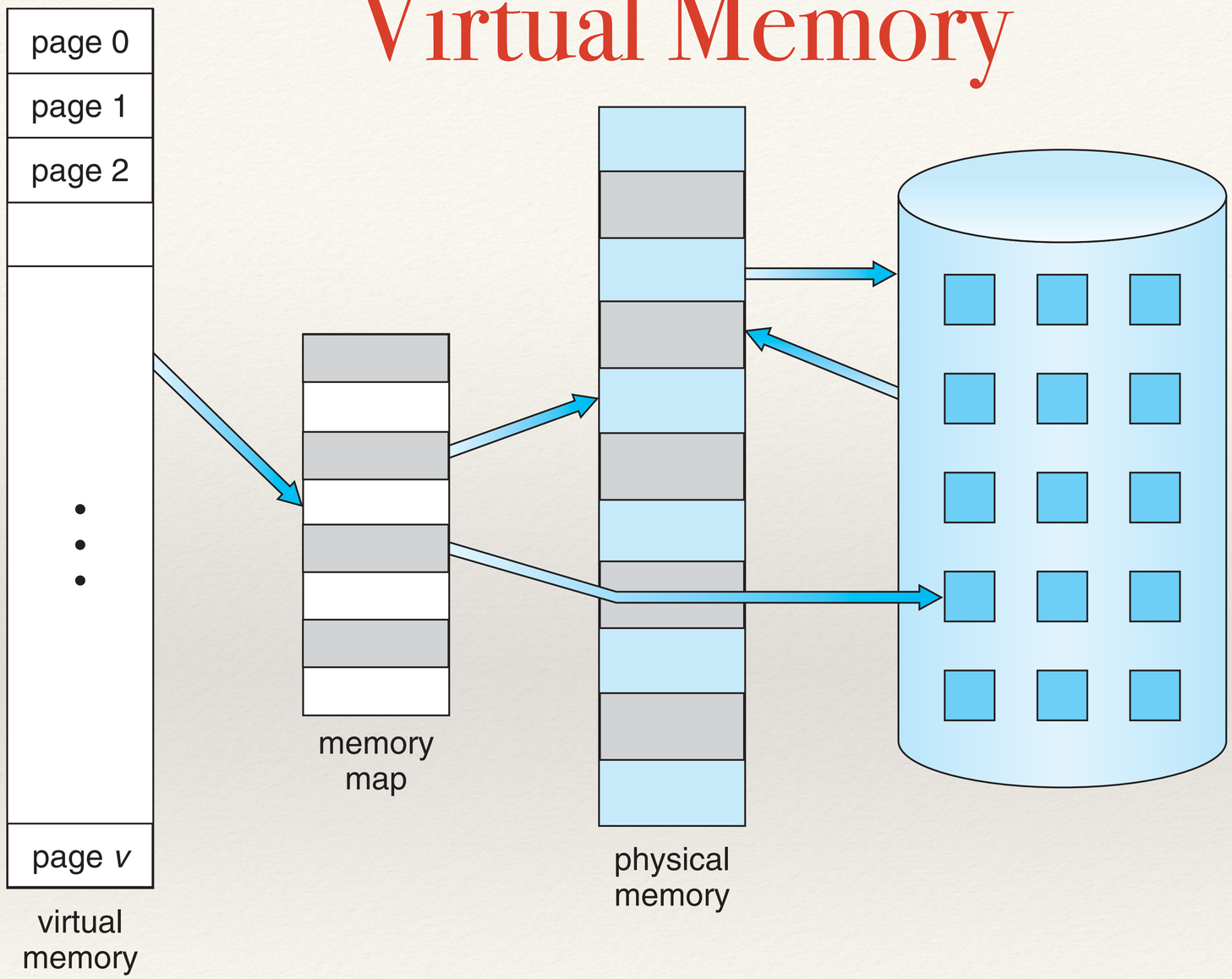
Background

- What sort of **advantages** are there, if we don't have to load the entire program into memory?
  - Can run more programs (because each program would take less physical memory)
  - Less I/O would be needed
    - Don't need the 'whole' user program into memory, so each user program would run faster.
  - Can run programs that are larger than the physical memory

Virtual Memory

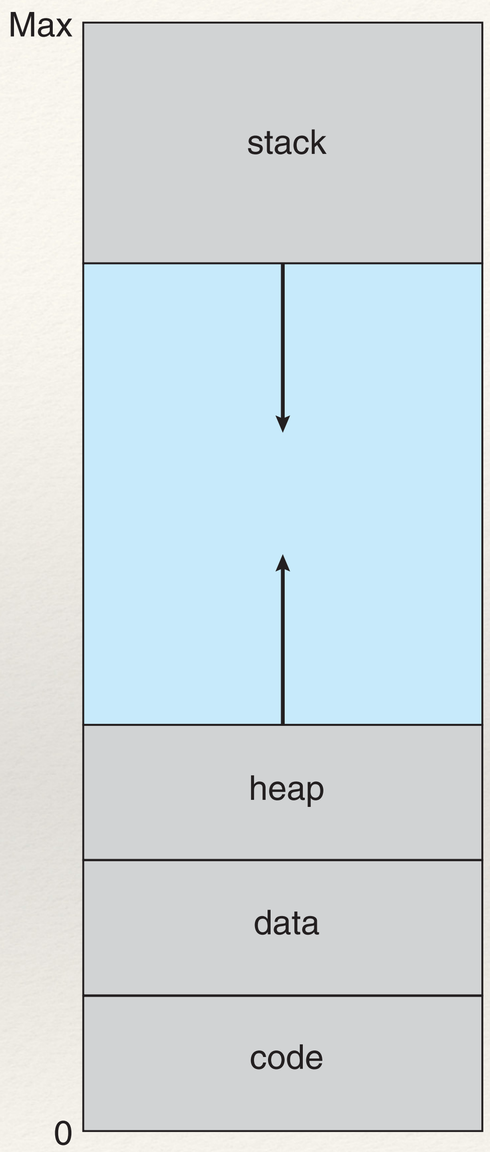
- Virtual memory involves the separation of logical memory as perceived by the user from physical memory.
- The programmer no longer needs to worry about the limited size of physical memory and can focus on the programming task.
- The Virtual address space refers to the logical (or virtual) view of how a process is stored in memory.

Virtual Memory



Note the virtual memory is larger than the physical memory

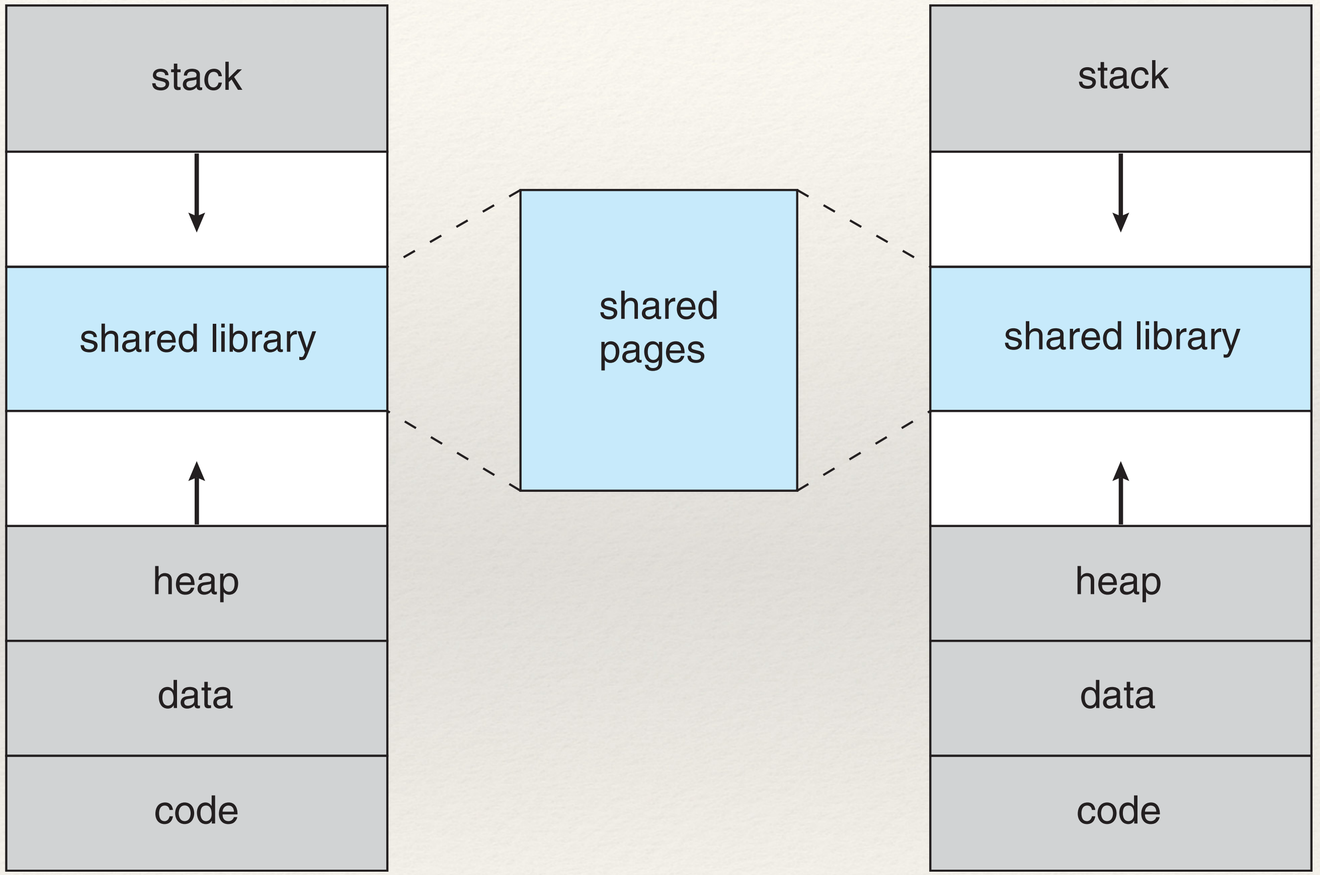
Virtual Memory



Virtual Memory

- Virtual memory allows files and memory to be shared by two or more processes. This creates some of the following advantages
  - System libraries can be shared by multiple processes. The processes treat the libraries as part of their own memory, even though they are shared
  - Processes can share memory (e.g., shared memory regions)
  - Pages can be shared during process creation with the fork() system call

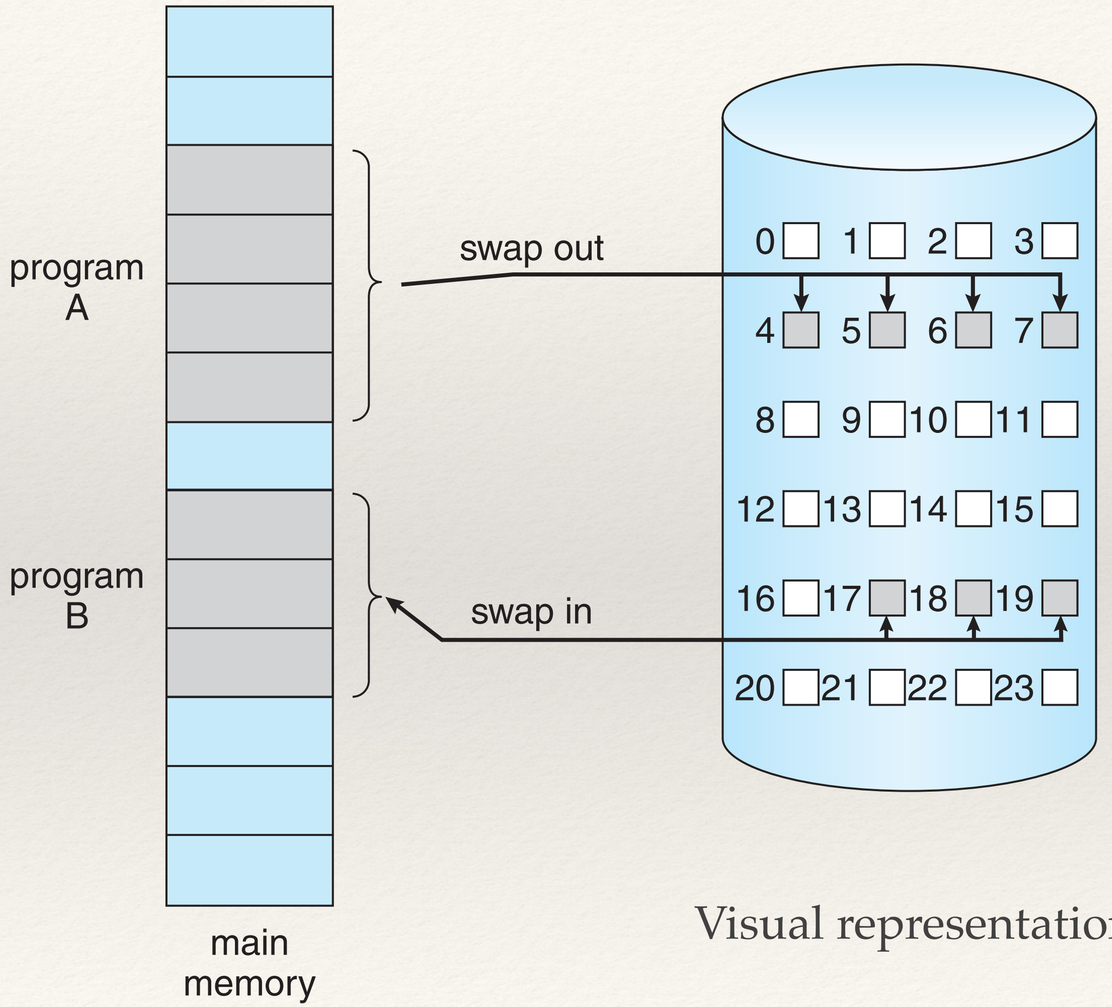
Virtual Memory



Demand Paging

- Demand paging involves only loading the pages into memory as they are needed (i.e., not the entire program at load time).
- Pages that are never accessed, are never loaded into the system.

Demand Paging



Visual representation of pages swapping in and out

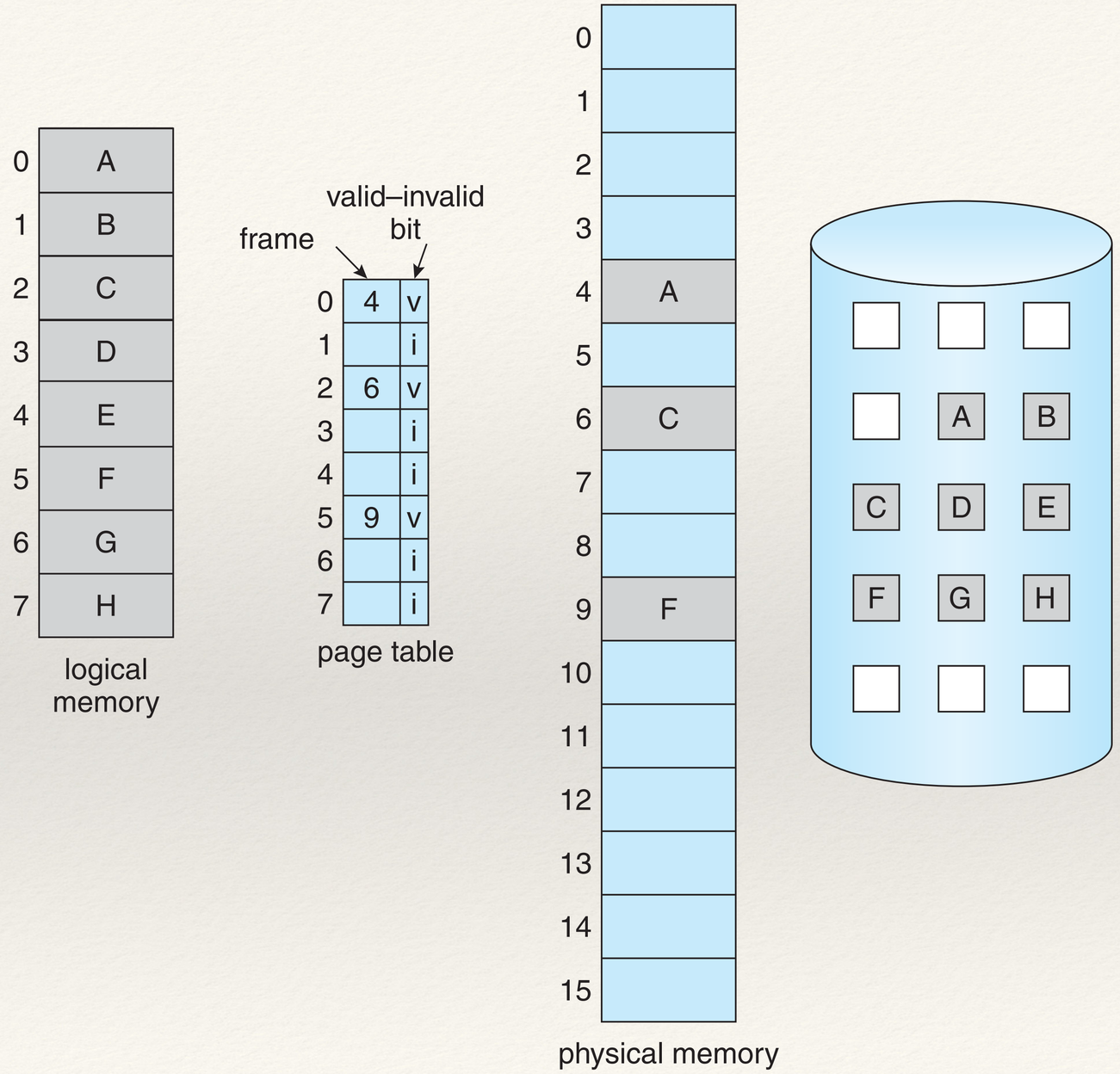
Demand Paging

- Lazy swapper - never swaps a page into memory unless it is needed.
  - Technically it would be a lazy pager, as a pager involves individual pages of a process and not the whole process.

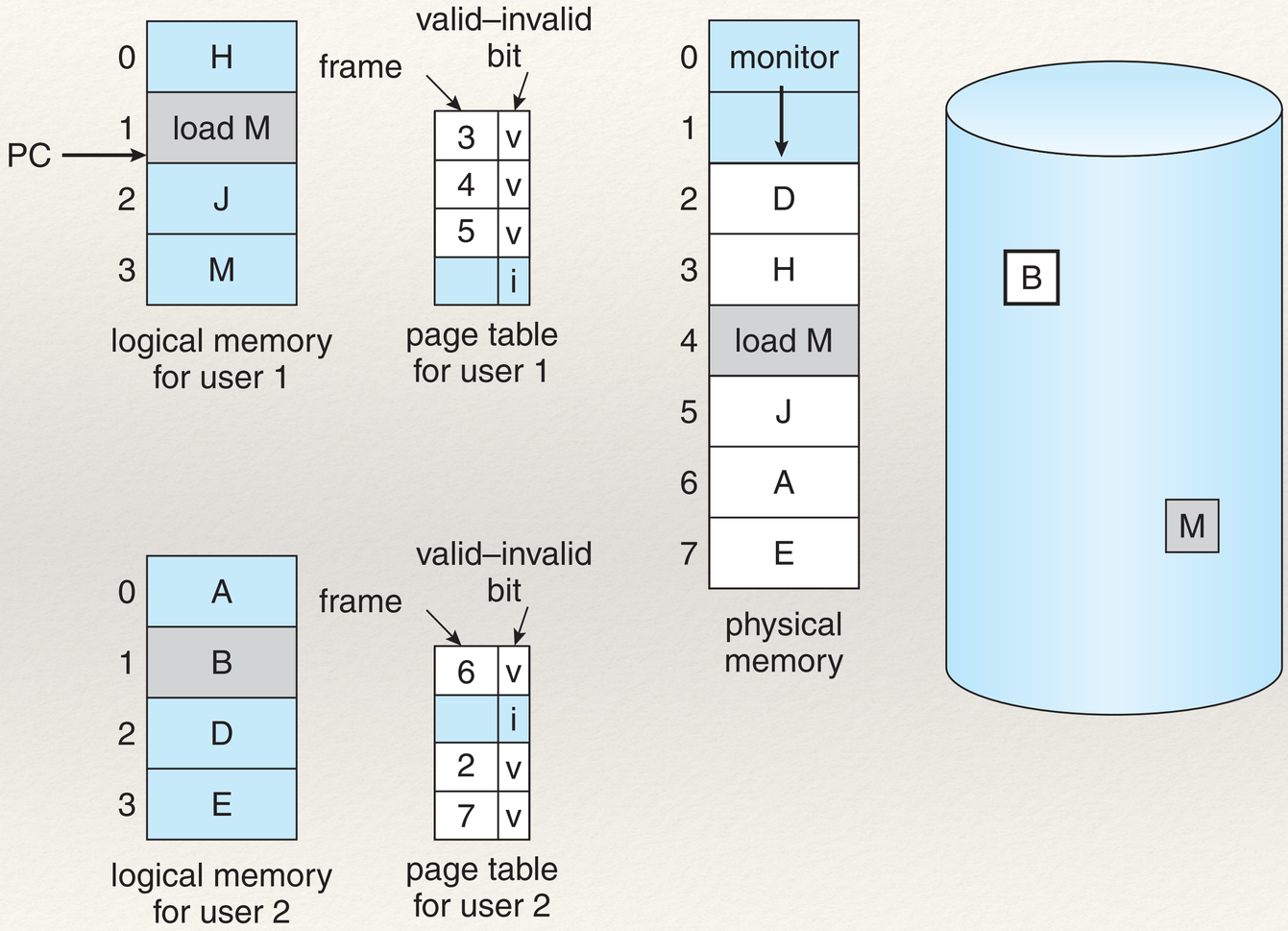
Demand Paging

- How does the memory manager know if it's in physical memory or the backup store
- The pages can be marked as:
  - Valid (in physical memory)
  - Invalid (not in physical memory)

Demand Paging



Demand Paging



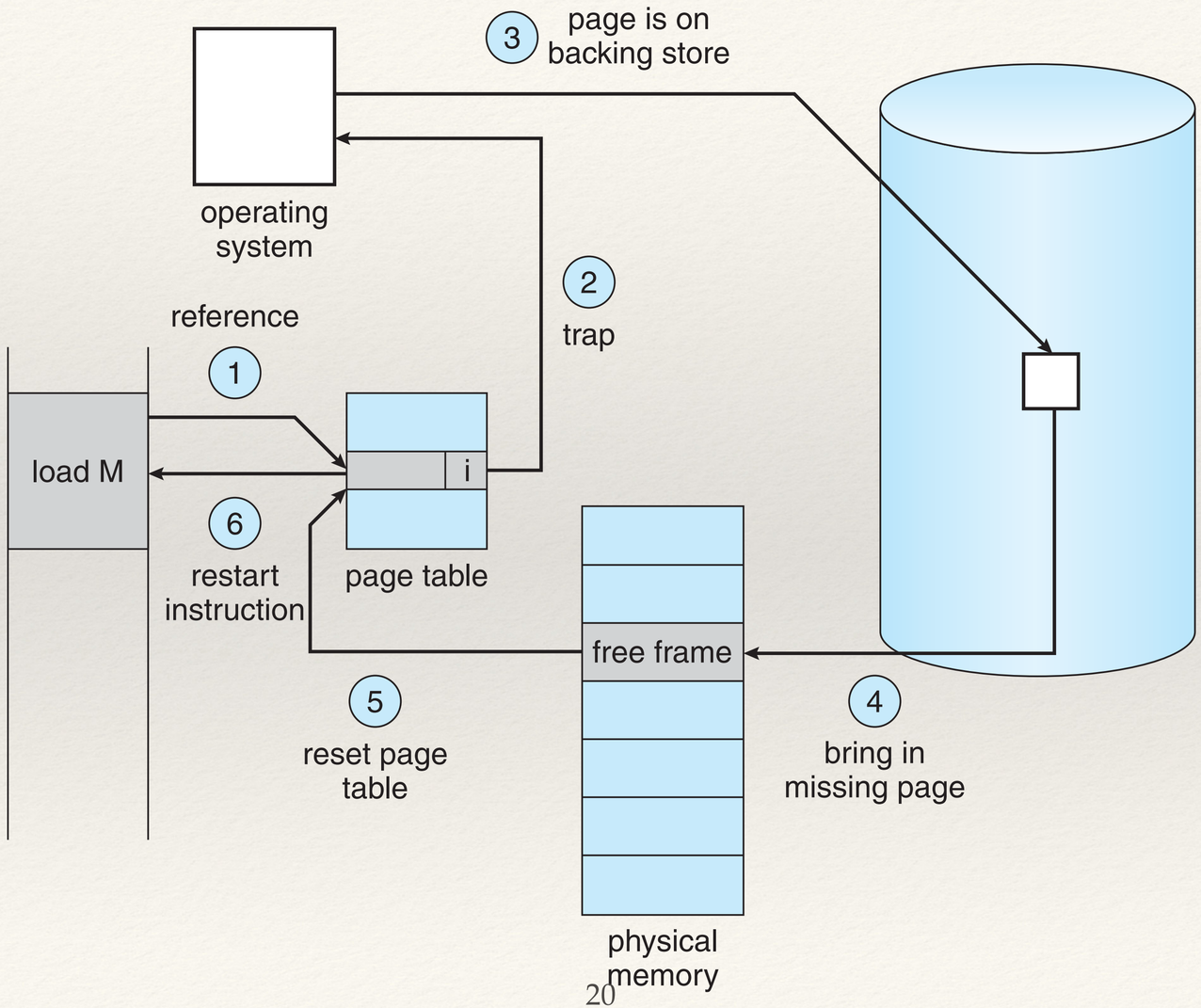
Demand Paging

- What happens if a page is needed and it's not in memory?
  - A page fault occurs

Demand Paging

- The page fault involves:
  - Was it a valid memory address? If no, terminate process
  - If it was valid, find a free frame, schedule the disk to load directly into the free frame
  - When the disk read is complete, modify the internal table to mark that page has valid
  - Restart the instruction that caused the page fault initially

Demand Paging



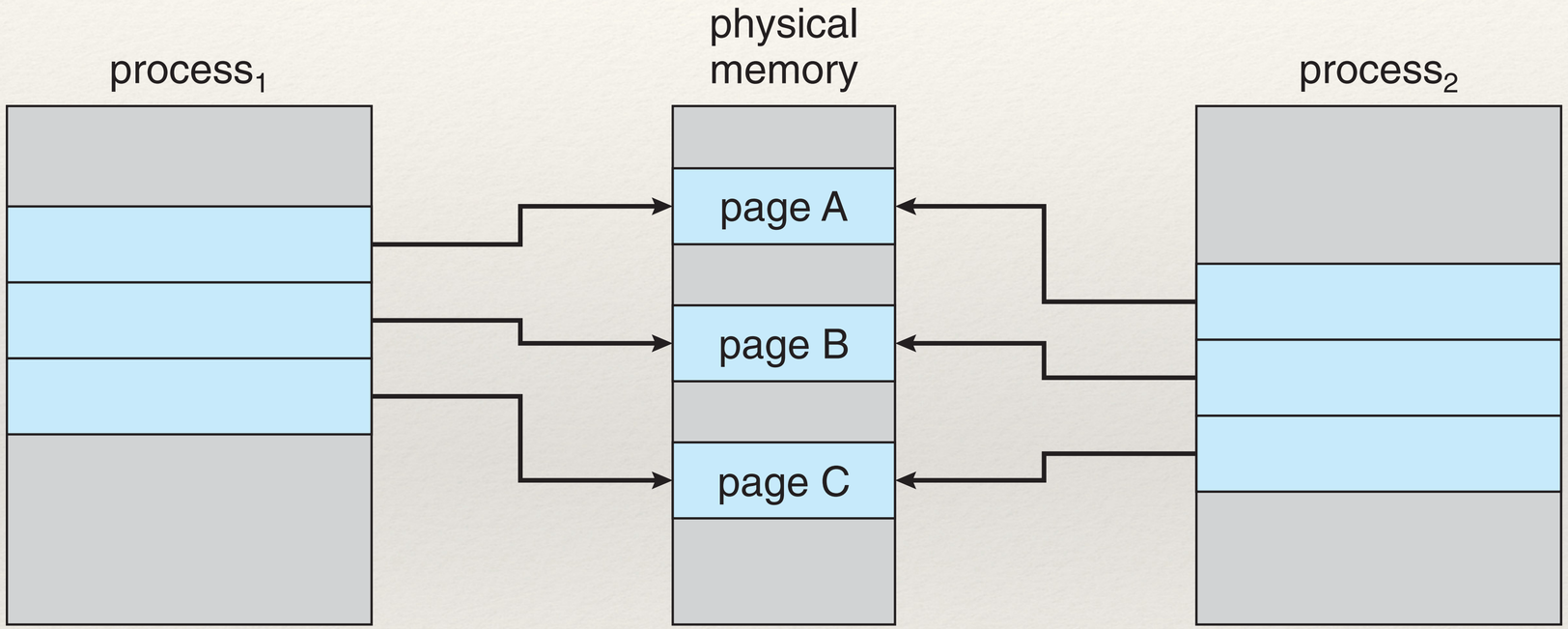
Demand Paging

- Pure demand paging - is a term used in which a page is never brought into memory until it is required

Copy-on-Write

- A page can be initially shared by two processes (think fork())
- The page is only copied, once it needs to be written

Copy-on-Write



Copy-on-Write